#5 If the parent has lesser priority than the current element p...
#6 ... then moves the parent one position down (implicitly p goes one up)
#7 Otherwise, we have found the right place for `current` and we can exit the loop
#8 At this point, `parentIndex` is the right position for `current`

Now, this can be done efficiently, in the same way each iteration of the *insertion sort* algorithm works: we initially save in a temporary variable a copy of the element to bubble up (call it `X`), and then check the elements on its left in the array; we "move" the elements to their right (check figure 2.7.C) by copying them to the next position on their left, until we find an element with a priority higher than X's. This can be done with just (at most) `H+1` assignments for a path of length `H`, thus saving about 66% of the assignments.

Listing 2.3 shows the improved version of the `bubbleUp` method: notice how, at some point, there will momentarily be two copies of the element originally at index `[3]` (and later, of the element at index `[1]`): this is because instead of actually swapping elements, we can just overwrite current element with its parent at each iteration (line #6) and only at the end we write the element bubbling up just once, as the last action in the method. This works because bubbling up an element is conceptually equivalent to the inner loop of *insertion sort*: we are looking for the right position to insert current element into the path connecting it to heap's root.

2.6.2 PushDown